

O que não coube na tela, *por escrito*

Isto aqui é o aprofundamento do bloco da tarde. Serve para você refazer sozinho, na segunda-feira, tudo que a gente fez junto — e para terminar as partes que ficaram pela metade.

Ordem sugerida de leitura: seção 1 antes de qualquer teste, seção 2 se a instalação travou, seção 7 para fechar o exercício. O resto é consulta.

01	Plano de mensuração	o aprofundamento — etapas, eventos, KPI e guardrail
02	Instalação passo a passo	Mac e Windows, o kit de seis comandos, e o que fazer quando trava
03	CLAUDE.md e CRO.md	os dois arquivos que o Claude lê antes da sua primeira pergunta
04	Os três hooks	instalar, testar e desligar os guardrails de processo
05	Referências de hooks	os cinco momentos em que um hook pode entrar
06	Bônus por escrito	Cowork, tarefas agendadas, custo, memória, base do time
07	O exercício do nó 12	o seu processo vira uma skill — para terminar em casa
08	Links	plugin, MCPs, extensão, documentação

Plano de mensuração, o aprofundamento

É o que precisa estar pronto antes de pensar em teste A/B.

Não adianta colocar IA numa operação que está quebrada. Shit in, shit out.

Por que isto vem antes de qualquer teste

Um teste A/B só existe porque alguém consegue contar duas coisas: quantas pessoas viram e quantas converteram. Se a contagem está errada, o teste não fica "menos preciso" — ele fica indecifrável. Você vê um número, não consegue explicar, e chuta.

A checagem de sempre, antes de tudo: o `add_to_cart` bate com o carrinho? O `purchase` bate com a plataforma? Se não bate, o teste nasce morto — e não existe ferramenta que conserte.

Essa é também a razão do hook `sem-plano-sem-hipotese.sh` (seção 4): ele não deixa você gerar hipótese num projeto que ainda não tem plano de mensuração escrito.

Passo 1 — dividir a jornada em etapas

As etapas são as que **você acabou de percorrer** quando fez a jornada do seu usuário. Entrou no site, procurou o produto, colocou no carrinho, foi até o fim. Cada tela onde o usuário pode desistir é uma etapa.

Escreva com o nome que o seu time já usa internamente, não com o nome do relatório da ferramenta. Se internamente vocês dizem "escolha de tamanho", a etapa se chama escolha de tamanho.

Exemplo de decomposição — loja de moda

1. Descoberta (anúncio, busca, e-mail) · 2. Listagem · 3. Página de produto · 4. Escolha de tamanho · 5. Carrinho · 6. Identificação e frete · 7. Pagamento · 8. Pós-compra

Uma forma rápida de checar se a lista está completa: monte o funil de trás para frente. Qual é a macroconversão? Qual é a etapa que antecede ela? E a que antecede essa?

Passo 2 — um evento por etapa, e onde se lê

Cada etapa precisa de exatamente um evento que diga "aconteceu". Não dois, não meio. E precisa de um lugar onde você lê o número — a ferramenta, o relatório, a tabela.

A coluna "Onde mede" é a que mais gente pula, e é a que evita a discussão de terça-feira sobre qual número está certo. Se dois lugares medem a mesma etapa, escolha um como fonte de verdade e anote o outro como conferência.

Passo 3 — KPI primário e guardrail

O **KPI primário** é o número que decide se o teste ganhou. Um só. O **guardrail** é o número que você não aceita piorar em troca — mesmo que o KPI suba.

Sem guardrail, todo teste "ganha": dá para subir add-to-cart aumentando devolução, subir lead baixando qualidade, subir conversão do checkout escondendo o frete. O guardrail é o que impede você de comemorar um estrago.

KPI primário

Conversão sessão → pedido. Decide o teste.
Definido antes do Start, não depois de olhar o resultado.

Guardrail

Taxa de troca por tamanho. Se subir, o ganho de pedidos não vale — e a decisão vira "investigar", não "implementar".

Template preenchível

É a mesma tabela da seção 3 do seu **CRO.md**. Preencha aqui no papel e depois transcreva — ou preencha direto no arquivo, que é onde ele vive.

KPI primário: _____ Guardrail: _____

ETAPA	EVENTO	ONDE MEDE

Um exemplo preenchido — Malha Viva **LOJA FICTÍCIA**

KPI primário: conversão sessão → pedido • Guardrail: devoluções (refund)

ETAPA	EVENTO	ONDE MEDE
Busca / home	view_search_results	GA4
Listagem	view_item_list • select_item	GA4
Página de produto	view_item • add_to_cart	GA4
Carrinho	view_cart	GA4
Checkout	begin_checkout • add_shipping_info • add_payment_info • purchase	GA4 + backoffice
Pós-compra	refund	GA4 — é o guardrail

Repare em duas coisas. A última linha existe só para o guardrail: sem ela, você não tem como saber se o teste que subiu pedidos subiu também a devolução. E **purchase** tem duas fontes de propósito — GA4 decide o teste, o backoffice confere se o GA4 está contando certo.

Os erros comuns

1 • Medir clique em vez de resultado

"Cliques no botão de frete" não é etapa da jornada: é reação a um elemento. O usuário pode clicar três vezes porque não entendeu. Meça o que ele completou — chegou na próxima etapa — e use o clique como diagnóstico, nunca como KPI.

2 • KPI sem guardrail

Um número sozinho sempre pode ser melhorado à custa de outro. Se você não consegue nomear o que não pode piorar, você ainda não entendeu o que o teste está trocando.

3 • Evento sem dono

Evento que ninguém mantém quebra na próxima publicação do site e ninguém percebe por três meses. Cada linha da tabela precisa de um nome de pessoa atrás — quem implementa, quem confere que ainda dispara.

Se uma linha da sua tabela não passa nos três, ela não está pronta. Voltar aqui é mais barato que ler um resultado que você não consegue explicar.

Instalação passo a passo

Mac e Windows. Sem Node, sem npm.

Passo 1 — instalar o Claude Code

Abra o terminal (no Mac: **Cmd + Espaço**, digite Terminal, Enter. No Windows: tecla Windows, digite PowerShell, Enter) e cole a linha do seu sistema.

MAC • TERMINAL

```
curl -fsSL https://claude.ai/install.sh | bash
```

WINDOWS • POWERSHELL

```
irm https://claude.ai/install.ps1 | iex
```

Passo 2 — abrir e entrar

```
claude
```

Esse comando abre o login no navegador. Use a mesma conta Claude Pro do bloco da manhã. Se o navegador não abrir sozinho, copie a URL que apareceu no terminal e cole no navegador.

Passo 3 — o kit do curso, seis linhas

Uma pasta para a loja, e o kit dentro do Claude. Rode **fora** do Claude Code — se você estiver dentro dele, digite **/exit** antes. Tudo que roda com **/cro:** na frente vem daqui.

TERMINAL • MAC E WINDOWS • FORA DO CLAUDE

```
mkdir minha-loja
cd minha-loja
claude plugin marketplace add lucianfialho/cro-plugin
claude plugin install cro@cro-plugin
claude plugin marketplace add anthropics/claude-plugins-official
claude plugin install skill-creator@claude-plugins-official
```

Seis linhas, uma vez na vida. O esperado é uma mensagem com ✓ em cada uma das quatro linhas de plugin. Para conferir depois:

```
claude plugin list
claude plugin details cro@cro-plugin # deve mostrar Skills (10)
```

Quando der errado

O QUE APARECE NA TELA

O QUE FAZER

command not found: claude (mac), ou uma frase com **claude** e "**não é reconhecido**" (win)

O instalador terminou, mas o terminal que está aberto não sabe disso. **Feche a janela do terminal e abra de novo.** Fechar é clicar no X (ou **Cmd+Q** no Mac) — minimizar não conta.

O Windows bloqueia o script — uma frase com "**execution of scripts is disabled**" ou **ExecutionPolicy**

Feche o PowerShell, clique com o botão direito no ícone dele e escolha **Executar como administrador**. Rode **Set-ExecutionPolicy RemoteSigned**, confirme com **S** (ou **Y**) e rode a linha de instalação de novo.

O QUE APARECE NA TELA

O QUE FAZER

O plugin não aparece: você digita `/` e não vê `/cro:` na lista, mesmo tendo visto a mensagem de sucesso

Plugin só entra na próxima abertura. **Saia e entre de novo:** digite `/exit`, espere o cursor do terminal voltar, digite `claude` e confira de novo.

Você digitou um comando `claude plugin ...` e o **Claude respondeu em texto**, explicando o comando

Você está *dentro* do Claude Code, e ali aquilo não é comando, é pergunta para o modelo. Nada foi instalado. `/exit` e rode de novo no terminal.

Uma frase com **"Failed to clone marketplace repository"**

Falta o git nessa máquina. Faça este passo em dupla com alguém que tenha, e instale o git depois, em casa.

Nenhuma linha acima bate com o que você está vendo

Não insista sozinho. Chame o facilitador e mostre a tela.

Regra geral das três saídas: comando não encontrado → feche e abra o terminal. Windows reclamando de script → PowerShell como administrador. Plugin invisível → saia e entre no Claude de novo. Quase tudo cai numa dessas.

CLAUDE.md e CRO.md

O Claude só sabe o que você deu pra ele saber.

CLAUDE.md — a porta de entrada

A primeira coisa que ele lê, toda sessão, antes da sua primeira mensagem. Diz quem você é, o que este projeto é, o que ele nunca deve fazer — e importa os outros arquivos.

CRO.md — a loja explicada

Convenção do nosso workshop, não spec de mercado. É a loja explicada para quem nunca entrou nela: jornada, mensuração, segmentos, voz do cliente.

Como criar os dois, em ordem

FAÇA AGORA • DENTRO DA PASTA DA LOJA

```
cClaude  
/cro:cro-md
```

O `/cro:cro-md` é a skill do plugin: pergunta o nome da loja, cria o `CRO.md` a partir do template e escreve o `CLAUDE.md` com a linha `@CRO.md`. Se o `CLAUDE.md` já existe, só acrescenta a linha. Abra o `CLAUDE.md` depois: é isso que ele vai ler toda vez.

E o `/init`? É o comando nativo que escreve o `CLAUDE.md` sozinho *num projeto de código*: ele lê o repositório e documenta comandos e arquitetura. Numa pasta vazia, escreve em inglês sobre software que não existe. Por isso no workshop a skill faz esse papel.

As 8 seções do CRO.md

#	SEÇÃO	O QUE VAI NELA	QUANDO
01	Negócio	Produto, ticket médio e o funil inteiro em uma linha.	pra casa
02	Jornada em etapas	As etapas que o usuário percorre, na ordem, com o nome que você usa internamente.	AO VIVO
03	Plano de mensuração	KPI primário, guardrail, e a tabela etapa/evento/onde mede da seção 1 deste PDF.	AO VIVO
04	Segmentos que importam	Três ou quatro recortes que se comportam diferente — não demografia genérica.	pra casa
05	Voz do cliente	O que o SAC ouve e as objeções que vendas escuta, sempre com a fonte do dado.	AO VIVO
06	Perfil demográfico	Quem <i>já</i> compra, puxado da sua própria base — não o público que você gostaria de ter.	pra casa
07	Testes e backlog	Testes já rodados com resultado, e as hipóteses na fila no template da Taciana.	pra casa
08	Design	Uma linha só: o contexto visual mora no <code>DESIGN.md</code> .	pra casa

Três a gente preencheu junto. As outras cinco já estão no template, com a instrução em comentário e um exemplo preenchido da Malha Viva para você trocar pelo seu.

O `CRO.md` serve primeiro para **você**: se não consegue escrever a seção, é porque ainda não sabe. Depois serve para o Claude, que lê tudo isso antes de qualquer análise, hipótese ou variante.

Como o @import funciona

Uma linha com @arquivo dentro do CLAUDE.md expande o conteúdo daquele arquivo na abertura da sessão. Você não pede, não cola, não lembra: ele já entra sabendo.

CLAUDE.MD — NA PASTA DA LOJA

```
# Projeto: loja Malha Viva
```

```
Contexto de CRO: @CRO.md
```

```
Design system: @DESIGN.md
```

CRO.md diz **como a loja converte**. DESIGN.md diz **como a loja parece**. Abriu o terminal na pasta, rodou claude: ele já sabe os dois.

A regra

Curto. O detalhe vai nos importados. O CLAUDE.md é índice, não enciclopédia: quem você é, o que é o projeto, o que nunca fazer, e os imports. Se ele cresceu, quebre em dois e importe.

E o sinal de que faltou contexto: você explicou a mesma coisa duas vezes para o Claude. Era pra estar num arquivo.

Os três hooks

Não é prompt. É código — não depende de o modelo lembrar.

Os três são **PreToolUse**: rodam antes da ferramenta, e cada um é a etapa anterior cobrando a conta. Eles não são de segurança, são de **processo**: o Claude não consegue pular uma etapa do ciclo porque um script bloqueia.

HOOKE	BLOQUEIA	EXIGE
sem-plano-sem-hipotese.sh	a skill hipotese-estruturada	CRO.md com a seção "Plano de mensuração" preenchida
sem-srm-sem-resultado.sh	comandos que leem resultado de teste A/B (converted, conversão, lift, cro-clean.db)	a skill srm-check ter dado  e gravado .cro/srm-ok
sem-design-sem-variante.sh	a skill variante-builder e a escrita de variante*.html/.css/.js	DESIGN.md no projeto

Quando bloqueia, a mensagem diz o que falta e como resolver. Não é sermão, é o caminho de volta.

Instalar em 3 passos

Os arquivos estão na pasta **hooks/** de github.com/lucianfialho/cro-plugin. Baixe pelo botão **Code** → **Download ZIP**, descompacte, e abra o terminal na pasta descompactada (os comandos abaixo assumem que ela se chama **cro-plugin**).

1. Copie os três **.sh** para a pasta de hooks do Claude:

```
mkdir -p ~/.claude/hooks
cp cro-plugin/hooks/sem-plano-sem-hipotese.sh ~/.claude/hooks/
cp cro-plugin/hooks/sem-srm-sem-resultado.sh ~/.claude/hooks/
cp cro-plugin/hooks/sem-design-sem-variante.sh ~/.claude/hooks/
```

2. Registre os hooks. Se você **não tem** **~/.claude/settings.json**, copie o da pasta:

```
cp cro-plugin/hooks/settings.json ~/.claude/settings.json
```

Se você **já tem** um **settings.json**, não sobrescreva: abra os dois e cole os três blocos de dentro de **"PreToolUse"** no array **"PreToolUse"** que já existe no seu arquivo. Se o seu não tem **"hooks"**, cole o objeto **"hooks"** inteiro.

3. Reinicie o Claude Code. Hooks são lidos na abertura da sessão — se você editar o **settings.json** com a sessão aberta, o hook antigo continua valendo.

Windows: funciona no Git Bash, que é o que o Claude Code usa para rodar hooks. Não precisa de **jq** nem de **chmod**.

Testar que está funcionando

Sem abrir o Claude, os dez casos de cada hook — bloqueio, liberação e regressão:

```
bash cro-plugin/hooks/test.sh # deve terminar com FAIL: 0
```

Com o Claude aberto, num diretório vazio, peça "Rode a skill hipótese-estruturada". Tem que aparecer BLOQUEADO pelo hook sem-plano-sem-hipotese. Depois copie o `aluno/CR0.md` para a pasta, preencha a seção "Plano de mensuração" e peça de novo: passa.

O que conta como preenchida: o hook lê da linha `## ... Plano de mensuração ...` até o próximo heading e descarta linha em branco, comentário HTML, cabeçalho da tabela e separador. O template recém-copiado, com a tabela vazia, ainda bloqueia — basta uma linha de dado de verdade para liberar.

Desligar um hook

1. **Só neste projeto, agora:** crie o arquivo que falta. É a saída pretendida.
2. **Temporariamente:** apague o bloco do hook em `~/.claude/settings.json` e reinicie o Claude Code.
3. **De vez:** apague o `.sh` de `~/.claude/hooks/` e também o bloco do `settings.json`.

Para conferir o que está ativo agora, rode `/hooks` dentro do Claude Code.

Referências de hooks

Os cinco momentos em que um script pode entrar.

Um hook é um script seu que o Claude Code executa num momento definido. Ele recebe um JSON no stdin com o contexto (cwd, hook_event_name, tool_name, tool_input). Para **bloquear**, o script escreve a mensagem no stderr e sai com código 2; sair com 0 deixa passar.

EVENTO	QUANDO DISPARA
SessionStart	Início ou retomada de sessão — bom para injetar contexto do dia ou conferir pré-requisitos do projeto.
UserPromptSubmit	Antes do Claude processar a sua mensagem — dá para acrescentar contexto ou barrar um pedido inteiro.
PreToolUse	Antes de executar qualquer ferramenta (Bash, Edit, Read) — é onde vivem os três hooks de CRO da seção 4.
PostToolUse	Após executar uma ferramenta com sucesso — bom para validar o que acabou de ser escrito, rodar lint, registrar log.
Stop	Quando o Claude termina de responder — bom para notificação, checklist final ou resumo da sessão.

Catálogo de hooks prontos

github.com/lucianfialho/awesome-claude-code-hooks

Exemplos organizados por evento, para copiar e adaptar. A documentação oficial completa está em code.claude.com/docs/en/hooks.

A regra para saber se algo deve virar hook: se você precisa que aconteça **sempre**, e não dá para aceitar que o modelo esqueça uma vez, é hook. Se é orientação que admite julgamento, é CLAUDE.md ou skill.

Bônus por escrito

O que a gente escolheu na hora — e o que não coube.

1 • Cwork — o mesmo Claude, sem terminal

Para quem do time não vai abrir o terminal. É um app de desktop que trabalha em pastas, documentos e planilhas: você descreve o resultado, ele executa.

- **mesma conta** — o Pro da manhã. Mesmas skills, mesmo modelo.
- **mesmo contexto** — o `CRO.md` e o `DESIGN.md` que você escreveu hoje servem lá também.
- **quando usar** — relatório, organização, análise de arquivo. Terminal quando precisa conectar ferramenta.

2 • Tarefas agendadas — o relatório que se escreve sozinho

```
cClaude
/loop --cron "0 8 * * MON" "Puxa o GA4 da semana com o gmp-cli,
compara com a anterior, escreve o resumo em relatorios/semana.md"
```

- `/loop` — repete num intervalo ou num cron. Roda numa sessão nova, na sua máquina.
- `/schedule` — uma vez, numa data e hora.
- **limite** — a máquina precisa estar ligada. A tarefa expira em 3 dias: agendou, confere.

3 • Custo — cinco comandos

Custo é contexto: quanto ele carrega, quanto ele pensa.

COMANDO	O QUE FAZ
<code>/usage</code>	quanto do seu limite você já gastou nesta janela
<code>/model</code>	Haiku, Sonnet, Opus, Fable. Começa no Sonnet: sobe se errou, desce se foi fácil.
<code>/effort</code>	quanto ele raciocina antes de responder. Baixo pra rotina, alto pra decisão.
<code>/compact</code>	resume a conversa e libera espaço sem perder o fio
<code>/context</code>	o que está ocupando a memória de trabalho agora

No plano Pro/Max você não paga por token: você gasta **janela de uso**. Sessão longa com Opus esgota a janela mais rápido.

4 • Memória — três memórias, três donos

CLAUDE.md

Você escreve. Regras e imports: `@CRO.md`, `@DESIGN.md`.

Memória automática

O Claude escreve pra ele mesmo, por projeto, em `~/cClaude/projects/.../memory`. Preferências, decisões, feedback.

Skills

O time escreve. Processo que não muda de sessão pra sessão.

Regra prática: se você explicou a mesma coisa duas vezes, isso era pra estar num arquivo.

5 · Base do time — ingestão de referências

Links, vídeos e PDFs virando base de conhecimento compartilhada:

1. **pasta** — `base/referencias.md`, importado no CLAUDE.md.
2. **skill** — `/ingere-referencia <url>`: lê, resume em 10 linhas, registra fonte e data, anexa na base.
3. **hábito** — achou um artigo bom, joga a URL. O Claude do time inteiro passa a saber.

Essa skill você constrói com o `/skill-creator:skill-creator` em 10 minutos. É a mesma receita do nó 9: ingredientes e modo de preparo.

O seu processo vira uma skill

Para terminar em casa, com a nota na mão.

Uma skill é uma receita: ingredientes e modo de preparo. No nó 9 os ingredientes eram dados de uma página. Aqui os ingredientes são o time da sua empresa.

Passo 1 — descrever o processo

Quatro coisas, e nenhuma delas é ferramenta:

Papéis

Quem coleta, quem analisa, quem prioriza, quem sobe o teste. Com o nome do cargo, não "o time".

Handoffs

Quem entrega para quem, em que momento. A passagem entre dois papéis.

Entregáveis

O que sai de cada etapa das 7 do ciclo da Taciana. O artefato, não a intenção.

Fontes

De onde vem o dado de cada etapa, **com nome**. "GA4, propriedade X", não "os dados da empresa".

DENTRO DO CLAUDE

```
/skill-creator:skill-creator
```

Quero uma skill com o processo de CRO da minha empresa. Me pergunta o que precisar. Papéis: ... Handoffs: ...

Não descreva a ferramenta. Descreva quem faz o quê. Se você travar numa seção, é a mesma lição do CRO.md: ainda não sabe.

Passo 2 — validar e sair sabendo a nota

```
/cro:valida-skill-cro
```

```
~/.claude/skills/meu-processo-cro/SKILL.md
```

Cada critério recebe **0, 1 ou 2**. A soma é a nota, de 0 a 10. E para cada nota abaixo de 2 vem a pergunta exata que a sua skill não responde — não "está vago".

#	CRITÉRIO	A PERGUNTA-GUIA
1	Papéis	A skill nomeia os cargos ou papéis das pessoas envolvidas no processo?
2	Handoffs	A skill nomeia as passagens entre papéis — quem entrega para quem, e em que momento?
3	Entregáveis	A skill nomeia o entregável de cada etapa — o que sai de cada handoff?
4	Ingredientes	A skill diz de onde vem o dado que alimenta cada etapa, com fonte nomeada?
5	Modo de preparo	A sequência de passos é executável, ou é vaga ("analisar", "otimizar", "melhorar")?

ESCALA	O QUE SIGNIFICA
0 — ausente	Não aparece em nenhum lugar da skill.
1 — parcial	Aparece, mas genérico: cita "o time de CRO" sem nomear os cargos, ou "os dados" sem dizer de onde vêm.
2 — completo	Nomeado e específico o suficiente para outra pessoa executar sem perguntar de volta.
NOTA FINAL	LEITURA
8-10	Descreve um processo executável. Está pronta para virar a skill de execução do time.
4-7	Tem processo real por trás, mas com lacunas que impedem outra pessoa de rodar sem perguntar de volta.
0-3	O que foi descrito ainda não é um processo — ou é a operação de uma ferramenta.

Sinal de alerta — ferramenta em vez de processo. Passos claríssimos ("abra o relatório X, clique em Y, exporte Z") e zero em Papéis, Handoffs e Entregáveis não é empate: é o sinal de que você descreveu como usar uma ferramenta, não o processo de CRO da sua empresa. O validador diz isso na cara.

Ele também não preenche a lacuna por você. Se a skill não diz de onde vem o dado, ele aponta a ausência — não sugere uma fonte plausível. Você tem que sair sabendo o que falta no seu processo, não recebendo um processo inventado.

Passo 3 — plugar no cro-plugin

A pasta `skills/metodologia-exemplo/` do plugin é um exemplo genérico — o processo de um time fictício. É o encaixe da sua skill.

1. Chegue em 8-10 no validador. Abaixo disso, resolva primeiro a pergunta que ele apontou.
2. Substitua o conteúdo de `skills/metodologia-exemplo/` pela sua skill — ou renomeie a pasta para o nome do seu processo. **A pasta precisa ter o mesmo nome do campo `name` do frontmatter.**
3. Reinstale o plugin. O time inteiro passa a rodar a sua metodologia com um comando.

Links

Tudo que apareceu na tela, num lugar só.

O plugin de hoje

```
claude plugin marketplace add lucianfialho/cro-plugin
claude plugin install cro@cro-plugin
```

Dez skills por papel — setup, analista, QA, dev, metodologia — mais os três hooks e o template de CRO.md. Grátis e aberto.

MCPs oficiais que valem pra CRO

MCP	O QUE TRAZ	REPOSITÓRIO
Google Analytics oficial do Google	GA4 direto na conversa: relatório, funil, tempo real.	github.com/googleanalytics/google-analytics-mcp
Chrome DevTools oficial do Chrome	Performance, DOM, screenshot da página.	github.com/ChromeDevTools/chrome-devtools-mcp
Playwright Microsoft	Navegar, preencher, testar a variante como um usuário.	github.com/microsoft/playwright-mcp
Notion	Backlog de hipóteses e relatórios onde o time já vive.	github.com/makenotion/notion-mcp-server

Instala com `claude mcp add`; o README de cada um tem a linha pronta.

Ferramentas

FERRAMENTA	PARA QUÊ	ONDE
Claude in Chrome — a extensão	Depois de instalar: <code>claude --chrome</code> e ele enxerga as abas.	chromewebstore.google.com/detail/claude/fcoeoabgfenegjglbfffodgkbbkcdhcgfn
gmp-cli	GA4, GSC, Ads e BigQuery no terminal.	github.com/lucianfialho/gmp-cli
DESIGN.md — a especificação do Google	Formato aberto, com validador. Este material segue um.	github.com/google-labs-code/design.md
awesome-claude-code-hooks	Hooks prontos por evento, para copiar e adaptar.	github.com/lucianfialho/awesome-claude-code-hooks

Documentação oficial

Hooks

code.claude.com/docs/en/hooks

Skills

code.claude.com/docs/en/skills

Memória

code.claude.com/docs/en/memo

O próximo desafio não é usar o Claude. É transformá-lo em parte do seu processo de CRO.